Задача 1

relname	last_analyze	last_autoanalyze	n_mod_since_analyz
flights	[NULL]	6-10-31 15:45:11.536 +0300	0

relname	last_analyze	last_autoanalyze	n_mod_since_analyze
flights	2026-10-31 15:58:38.321 +0300	2026-10-31 15:45:11.536 +0300	0

n_mod_since_analyze показывает количество строк, изменённых с момента последнего анализа. Это главный индикатор накопленных изменений. last_analyze и last_autoanalyze показывают время последнего ручного и автоматического анализа соответственно. То есть по показателю n_mod_since_analyze я определил что статистика не устарела, т. к. он равен нулю.

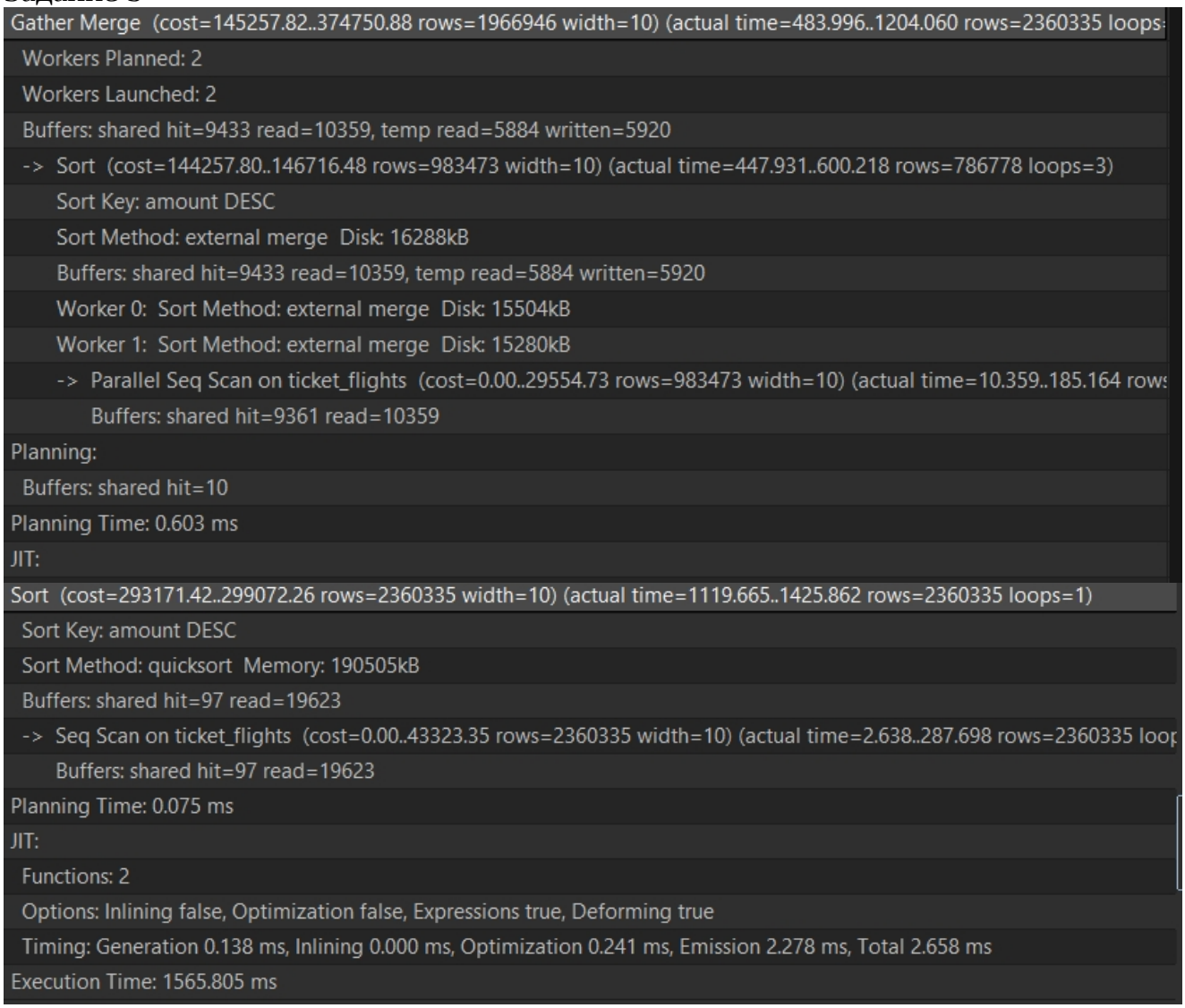
Задача 2

```
Hash Join  (cost=2026.92..51546.55 rows=603779 width=31) (actual time=22.231..1137.731 rows=678171 loops=1)
  Hash Cond: (tf.flight_id = f.flight_id)
  Buffers: shared hit=10129 read=10423
  ->  Seq Scan on ticket_flights tf  (cost=0.00..43323.35 rows=2360335 width=24) (actual time=0.107..523.434 rows=236033
        Buffers: shared hit=9297 read=10423
  ->  Hash  (cost=1816.96..1816.96 rows=16797 width=11) (actual time=21.863..21.865 rows=16835 loops=1)
        Buckets: 32768  Batches: 1  Memory Usage: 963kB
        Buffers: shared hit=832
        ->  Seq Scan on flights f  (cost=0.00..1816.96 rows=16797 width=11) (actual time=0.013..17.282 rows=16835 loops=1)
              Filter: ((scheduled_departure >= '2017-08-01 00:00:00+03'::timestamp with time zone) AND (scheduled_departure
              Rows Removed by Filter: 48829
              Buffers: shared hit=832
Planning:
  Buffers: shared hit=76 read=3 dirtied=3
Planning Time: 3.936 ms
Execution Time: 1180.079 ms
```

```
Hash Join  (cost=2026.92..51546.55 rows=603779 width=31) (actual time=10.829..783.044 rows=678171 loops=1)
  Hash Cond: (tf.flight_id = f.flight_id)
  Buffers: shared hit=10161 read=10391
  ->  Seq Scan on ticket_flights tf  (cost=0.00..43323.35 rows=2360335 width=24) (actual time=0.008..358.319 rows=236033
        Buffers: shared hit=9329 read=10391
  ->  Hash  (cost=1816.96..1816.96 rows=16797 width=11) (actual time=10.778..10.780 rows=16835 loops=1)
        Buckets: 32768  Batches: 1  Memory Usage: 963kB
        Buffers: shared hit=832
        ->  Seq Scan on flights f  (cost=0.00..1816.96 rows=16797 width=11) (actual time=0.007..7.920 rows=16835 loops=1)
              Filter: ((scheduled_departure >= '2017-08-01 00:00:00+03'::timestamp with time zone) AND (scheduled_departure
              Rows Removed by Filter: 48829
              Buffers: shared hit=832
Planning:
  Buffers: shared hit=6
Planning Time: 0.263 ms
Execution Time: 814.518 ms
```

время выполнения уменьшилось, т к pg_prewarm загружает страницы данных с диска в shared_buffers, поэтому запросы попадают в память без физ. ввода-вывода

Задание 3



Выделил больше памяти и external merge Disk заменилось на quicksort Memory

Задание 4

A-Z relname	A-Z pg_size_pretty
idx_test_skew_brin	24 kB
idx_test_skew_btree	2208 kB

BRIN хранит только диапазон минимальных и максимальных значений для каждого блока (или группы блоков), а не индексную запись для каждой строки, как B-Tree.

B-Tree: OLTP-поиск по ID, точечные запросы, запросы по уникальным значениям.

BRIN: большие таблицы с естественным порядком (например, журналы по времени, данные датчиков), запросы диапазонов, где данные физически упорядочены.
для B-Tree уменьшение fillfactor (например, до 70) оставляет свободное пространство на страницах индекса, уменьшая page splits при вставке, подходит для сценариев с частыми обновлениями/вставками.

Задание 5

query	calls	total_ms	avg_ms	shared_blks_read	temp_blks_written
EXPLAIN (ANALYZE, BUFFERS)¶SELECT	1	1 211,35	1 211,35	19 725	5 919
SELECT c.relname,a.*,pg_catalog.pg_ge	1	0,63	0,63	1	0
SELECT c.oid,c.*,d.description,pg_catal	1	0,19	0,19	1	0
SELECT typinput=$2::regproc as is_arra	2	0,17	0,08	0	0
SELECT pg_stat_statements_reset()	1	0,08	0,08	0	0
SELECT n.nspname = ANY(current_sche	2	0,07	0,03	0	0

Если у запроса большое значение temp_blks_written, это означает, что операция сортировки/хеширования выгружалась на диск, эффективность низкая - рекомендуется увеличить work_mem для этого запроса или оптимизировать логику.

Задание 6
Практикум 1

state	count
active	1
idle	3

relname	last_analyze	last_autoanalyze	n_mod_since_analyze
ticket_flights	[NULL]	2026-11-13 16:05:11.727 +0300	0

Updated Rows	0
Execute time	0,355s
Start time	Fri Nov 13 16:06:49 MSK 2026
Finish time	Fri Nov 13 16:06:50 MSK 2026
Query	ANALYZE ticket_flights

Updated Rows	0
Execute time	0,015s
Start time	Fri Nov 13 16:07:08 MSK 2026
Finish time	Fri Nov 13 16:07:08 MSK 2026
Query	CREATE EXTENSION IF NOT EXISTS pg_prewarm

Updated Rows	0
Execute time	1,856s
Start time	Fri Nov 13 16:07:32 MSK 2026
Finish time	Fri Nov 13 16:07:33 MSK 2026
Query	CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx ON ticket_flights(flight_id, amount)

123 pg_prewarm
19 720

123 pg_prewarm
9 103

state показывает состояние бэкенда (active, idle, idle in transaction и т.д.). Перед проведением замеров необходимо убедиться, что нет длительных idle in transaction сессий, удерживающих ресурсы, иначе результаты тестирования будут искажены.

Практикум 2

A-Z relname	A-Z size
idx_btree_ts	21 MB

A-Z relname	A-Z size
idx_brin_ts	48 kB

A-Z QUERY PLAN
Gather (cost=1000.00..20346.57 rows=37209 width=31) (actual time=0.566..127.424 rows=33142 loops=1)
Workers Planned: 2
Workers Launched: 2
Buffers: shared hit=8334
-> Parallel Seq Scan on iot_logs (cost=0.00..15625.67 rows=15504 width=31) (actual time=0.018..114.979 rows=11047 loops=3)
Filter: (reading_ts >= (now() - '1 day'::interval))
Rows Removed by Filter: 322286
Buffers: shared hit=8334
Planning:
Buffers: shared hit=37 read=1 dirtied=2
Planning Time: 1.594 ms
Execution Time: 130.123 ms

pages_per_range определяет, сколько страниц данных покрывает одна запись индекса BRIN. Меньшее значение означает более мелкую гранулярность индекса и большую точность фильтрации, но размер индекса увеличивается; большее значение даёт меньший индекс, но при запросе приходится сканировать больше блоков.

Раздел 7
Практикум 5

variant	size	leaf_pages	avg_leaf_density	leaf_fragmentation
ff_default (ff=90)	4488 kB	557	88,33	49,91
ff_70 (ff=70)	5680 kB	705	69,82	49,93

avg_leaf_density: средняя заполненность листовых страниц индекса. Чем ниже fillfactor, тем ниже плотность, тем больше свободного пространства остаётся.
leaf_fragmentation: степень фрагментации листовых страниц. Более низкий fillfactor обычно уменьшает фрагментацию, вызванную page splits.
Сценарии с низким fillfactor: таблицы/индексы с частыми вставками или обновлениями, нуждающиеся в резервировании пространства для уменьшения разделения страниц.

Раздел 9

query	calls	total_time_ms	avg_time_ms	rows	shared_blks_hit	shared_blks_read	temp_blks_written
ALTER TABLE ONLY boarding_passes¶ ADD CON	1	15 634,33	15 634,33	0	6 979 971	611 487	0
ALTER TABLE ONLY ticket_flights¶ ADD CONSTRA	1	6 774,26	6 774,26	0	2 195	17 672	20 796
ALTER TABLE ONLY boarding_passes¶ ADD CON	1	4 918,59	4 918,59	0	2 315	11 776	16 689
INSERT INTO iot_logs (reading_ts, sensor_id, value)	1	4 321,96	4 321,96	1 000 000	3 180 017	1	1 709
EXPLAIN (ANALYZE, BUFFERS)¶SELECT * FROM tick	2	3 542,87	1 771,43	0	208	39 346	38 695
COPY ticket_flights (ticket_no, flight_id, fare_conditi	1	3 539,09	3 539,09	2 360 335	60 692	0	0
ALTER TABLE ONLY tickets¶ ADD CONSTRAINT ti	1	2 089,56	2 089,56	0	2 161	11 872	2 843
ALTER TABLE ONLY ticket_flights¶ ADD CONSTRA	1	1 947,52	1 947,52	0	13 403	20 406	6 632
COPY tickets (ticket_no, book_ref, passenger_id, pas	1	1 932,23	1 932,23	829 071	42 559	0	0
CREATE INDEX IF NOT EXISTS ticket_flights_flight_i	1	1 821,1	1 821,1	0	9 548	10 309	16 175

query	calls	shared_blks_hit	shared_blks_read	hit_ratio_pct
SELECT * FROM readme LIMIT $1	1	0	1	0
EXPLAIN (ANALYZE, BUFFERS)¶SELECT * FROM ticket_flights ORDER BY amount DESC	2	208	39 346	0,53
EXPLAIN (ANALYZE, BUFFERS)¶SELECT flight_id, amount FROM ticket_flights ORDER BY amount DESC	1	132	19 725	0,66
ALTER TABLE ONLY tickets¶ ADD CONSTRAINT tickets_book_ref_fkey FOREIGN KEY (book_ref) REFERI	1	198	17 713	1,11
ALTER TABLE ONLY ticket_flights¶ ADD CONSTRAINT ticket_flights_pkey PRIMARY KEY (ticket_no, flig	1	2 195	17 672	11,05
ALTER TABLE ONLY tickets¶ ADD CONSTRAINT tickets_pkey PRIMARY KEY (ticket_no)	1	2 161	11 872	15,4
CREATE DATABASE readme_to_recover TEMPLATE template0	1	156	853	15,46
ALTER TABLE ONLY boarding_passes¶ ADD CONSTRAINT boarding_passes_flight_id_boarding_no_ke	1	2 181	11 904	15,48
ALTER TABLE ONLY boarding_passes¶ ADD CONSTRAINT boarding_passes_flight_id_seat_no_key UNI	1	2 300	11 840	16,27
ALTER TABLE ONLY boarding_passes¶ ADD CONSTRAINT boarding_passes_pkey PRIMARY KEY (ticket	1	2 315	11 776	16,43

query	calls	shared_blks_hit	shared_blks_read	hit_ratio_pct
SELECT * FROM readme LIMIT $1	1	0	1	0
EXPLAIN (ANALYZE, BUFFERS)¶SELECT * FROM ticket_flights ORDER BY amount DESC	2	208	39 346	0,53
EXPLAIN (ANALYZE, BUFFERS)¶SELECT flight_id, amount FROM ticket_flights ORDER BY amount DESC	1	132	19 725	0,66
ALTER TABLE ONLY tickets¶ ADD CONSTRAINT tickets_book_ref_fkey FOREIGN KEY (book_ref) REFERI	1	198	17 713	1,11
ALTER TABLE ONLY ticket_flights¶ ADD CONSTRAINT ticket_flights_pkey PRIMARY KEY (ticket_no, flig	1	2 195	17 672	11,05
ALTER TABLE ONLY tickets¶ ADD CONSTRAINT tickets_pkey PRIMARY KEY (ticket_no)	1	2 161	11 872	15,4
CREATE DATABASE readme_to_recover TEMPLATE template0	1	156	853	15,46
ALTER TABLE ONLY boarding_passes¶ ADD CONSTRAINT boarding_passes_flight_id_boarding_no_ke	1	2 181	11 904	15,48
ALTER TABLE ONLY boarding_passes¶ ADD CONSTRAINT boarding_passes_flight_id_seat_no_key UNI	1	2 300	11 840	16,27
ALTER TABLE ONLY boarding_passes¶ ADD CONSTRAINT boarding_passes_pkey PRIMARY KEY (ticket	1	2 315	11 776	16,43

В топ-10 по общему времени выполнения попали в основном служебные запросы, сгенерированные при создании индексов и ограничений.
В топ анализов со сбросом на диск попали команды ALTER TABLE ... ADD CONSTRAINT – при проверке ограничений для больших таблиц выделенной спецпамяти не хватает, и СУБД сбрасывает промежуточные результаты сортировки/хеширования во временные файлы.
В список анализа запросов с низким Cache Hit Ratio попали запросы, которые читают много данных с диска и мало берут из кэша.

Теоретическое задание

Объект	Ключевые столбцы	Назначение	Подводные камни
pg_stat_user_tables	last_analyze, last_autoanalyze, n_mod_since_analyze, seq_scan, idx_scan, n_live_tup, n_dead_tup	Определение устаревания статистики, оценка соотношения сканирований	Счётчики являются накопительными, необходимо сравнивать разницу двух моментов времени; n_live_tup - приблизительное значение
pg_stat_activity	state, wait_event_type, wait_event, query, query_start	Просмотр активных сессий в реальном времени, диагностика блокировок/ожиданий	Это срез, а не история; query показывает только текущий или последний запрос
pg_stat_statements	queryid, calls, total_exec_time, mean_exec_time, shared_blks_hit/read, temp_blks_written, rows	Ранжирование медленных запросов, анализ кэша, выявление проблем с временными файлами	Требуется предварительная загрузка; литералы нормализуются; не собирает до загрузки
pg_class	relname, relkind, relpages, reltuples, pg_relation_size()	Просмотр метаданных таблиц/индексов, размеров	reltuples - приблизительное значение, обновляется после ANALYZE/VACUUM
pg_prewarm	pg_prewarm(regclass, mode, ...)	Ручной/автоматический прогрев таблиц в кэш	Прогретые данные могут быть вытеснены; эффективно только при пустом кэше
pgstattuple / pgstatindex	avg_leaf_density, leaf_fragmentation, leaf_pages, free_percent	Анализ степени раздутия таблиц/индексов, фрагментации	pgstattuple сканирует всю таблицу, на больших таблицах работает медленно
pg_stat_bgwriter (PG16)	checkpoints_timed/req, buffers_backend/checkpoint, buffers_alloc	Оценка фоновой нагрузки ввода-вывода, частоты контрольных точек	В PG16 часть счётчиков буферов перенесена в pg_stat_io
pg_stat_database	blks_hit, blks_read, tup_returned, tup_fetched, xact_commit/rollback	Общий коэффициент попадания в кэш базы данных, активность транзакций	Статистика на уровне базы данных, не может детализироваться до таблицы

в этом задании наиболее полезными являются pg_stat_user_tables (быстрая проверка устаревания статистики) и pg_stat_statements (обнаружение проблем с временными файлами). При следующей диагностике аналогичной среды в

первую очередь следует проверить: 1) нет ли в pg_stat_activity длительных ожиданий/блокировок; 2) есть ли в топе pg_stat_statements запросы с большими temp_blks_written; 3) не устарела ли статистика ключевых таблиц в pg_stat_user_tables.